TEHNOLOGII WEB

Lucrarea de laborator nr. 2

# Vite și JavaScript modern

Proiect: City Explorer

## 1. Despre lucrare

Lucrarea introduce limbajul JavaScript și modul în care datele unei aplicații web sunt descrise și prelucrate în cod, separat de pagina care le afișează.

Se parcurg noțiunile de bază ale limbajului, împărțirea codului în module, metodele de lucru cu liste și preluarea datelor de la un server. Lucrul se desfășoară într-un proiect creat cu **Vite** și **React**, care se păstrează până la finalul semestrului. Interfața React face obiectul lucrării următoare, astfel că rezultatele se urmăresc în consola browserului. Exemplele folosesc o listă de orașe, iar exercițiile aplică aceleași noțiuni pe datele proiectului.

**Punct de plecare:** Lucrarea este independentă de precedenta. Proiectul creat la secțiunea 2 este nou, iar macheta HTML nu este necesară pentru exercițiile de azi. Cine nu a terminat laboratorul 1 poate începe direct de aici.

## 2. Proiectul Vite

Până acum pagina se deschidea cu Live Server. Un proiect care folosește module și pachete instalate cu npm are nevoie de mai mult: un server care servește fișierele în timpul lucrului și un pas de împachetare pentru publicare. Ambele sunt asigurate de Vite (figura 1).

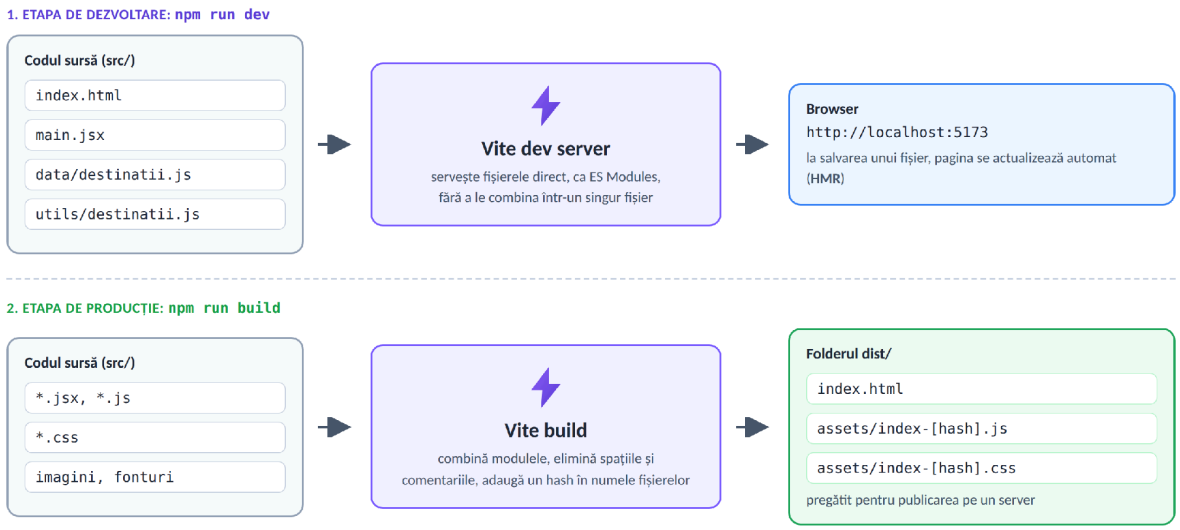



**Figura 1.** *Cele două moduri de funcționare ale instrumentului Vite.*

**Pasul 1.** În directorul `city-explorer`, cu terminalul deschis din **Terminal** → **New Terminal**, se execută:

```
npm create vite@latest
```

Se răspunde: numele `app`, framework-ul **React** și varianta **JavaScript**. Atenție la ultima întrebare: lista are mai multe opțiuni, implicit este selectat TypeScript, iar JavaScript este a treia, deci se coboară de două ori cu săgeata ↓ înainte de Enter (figura 2). La întrebarea finală, despre instalarea automată, se răspunde **No**.

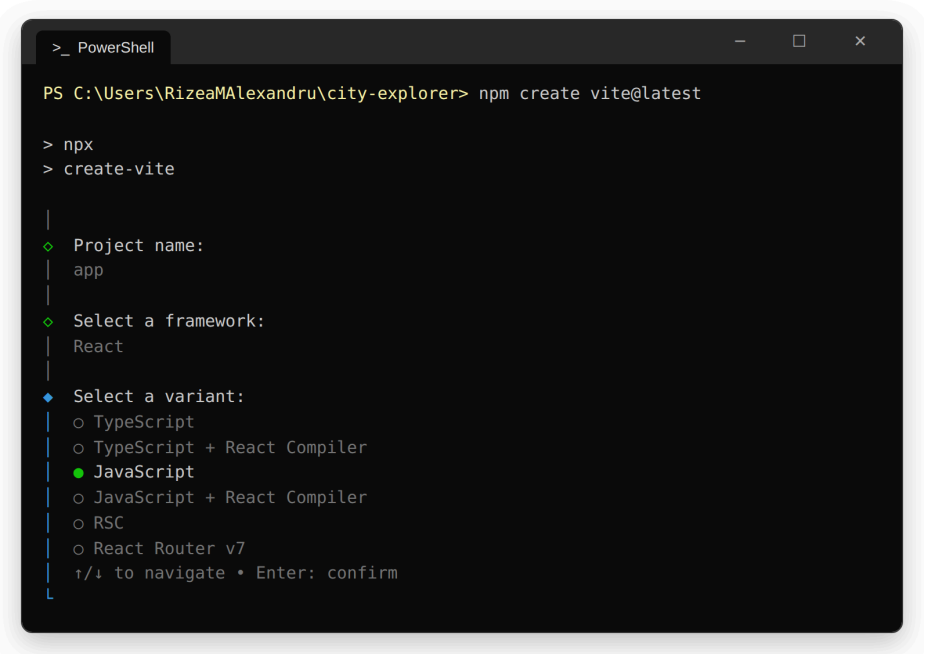



**Figura 2.** *Alegerea variantei JavaScript.*

**Pasul 2.** Se instalează pachetele și se pornește serverul (figura 3):

```
cd app
npm install
npm run dev
```

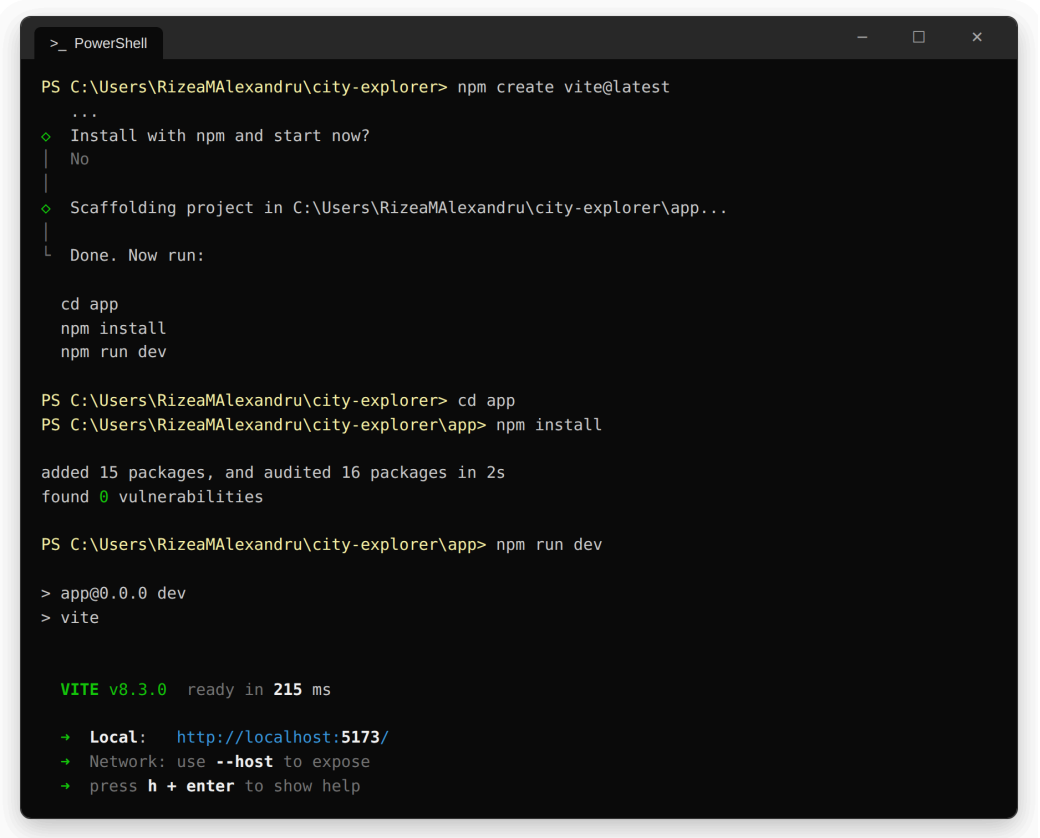



**Figura 3.** *Instalarea pachetelor și pornirea serverului.*

**Verificare.** Se deschide adresa afișată în terminal, `http://localhost:5173`. Apare pagina generată de Vite. Serverul rămâne pornit și ocupă terminalul; pentru alte comenzi se deschide un al doilea terminal, cu butonul **+**, iar serverul se oprește cu **Ctrl + C**.

Proiectul generat conține și un **linter**, un program care citește codul și semnalează greșelile obișnuite: variabile declarate și nefolosite, importuri inutile, apeluri scrise greșit. Configurația lui se află în fișierul `.oxlintrc.json`, iar verificarea se pornește oricând cu:

```
npm run lint
```

Rolul lui este de a prinde problemele înainte de a fi observate în browser. Mesajele apar în terminal, cu numele fișierului și numărul liniei. În alte proiecte se poate întâlni, cu aceeași funcție, instrumentul **ESLint**.

**Pasul 3.** Se face curățenie: se șterg directorul `src/assets` și fișierul `public/icons.svg`, iar în `src` se creează directoarele `data`, `utils` și `api`. Conținutul fișierului `src/App.jsx` se înlocuiește cu:

**src/App.jsx (se înlocuiește tot conținutul)**

```
export default function App() {
  return <h1>City Explorer</h1>
}
```

Fișierul `src/App.jsx` descrie ce se vede în pagină. Forma lui, numită JSX, și modul în care se construiesc componentele fac obiectul laboratorului următor; azi este suficient titlul de mai sus. Fișierele `src/App.css` și `src/index.css` se pot goli.

Codul scris în această lucrare nu produce interfață, ci rezultate afișate în consolă. El se apelează din `src/main.jsx`, unde se adaugă, la final, importurile și apelurile proprii:

**src/main.jsx**

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './index.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

// ===== Laboratorul 2 =====
console.log('L2 Merge!')
```

**Verificare.** În pagină apare titlul „City Explorer”, iar în consola browserului, deschisă cu **F12**, tab-ul **Console**, apare mesajul `L2 Merge!`. Rezultatele tuturor exercițiilor se citesc aici.

# 3. Noțiuni de bază de JavaScript

HTML descrie ce conține pagina, CSS cum arată, iar **JavaScript** ce face. Este un limbaj de programare care rulează în browser: ține date, le prelucrează și, mai târziu, schimbă pagina. Codul din această secțiune se poate scrie direct în consola browserului, deschisă cu **F12**, pentru a vedea imediat rezultatul.

## 3.1 Valori și variabile

O variabilă este un nume dat unei valori. Se declară cu `const`, dacă valoarea nu se mai schimbă, sau cu `let`, dacă se schimbă. Fiecare instrucțiune se poate încheia cu punct și virgulă, dar nu este obligatoriu.

```
const oras = 'Lisabona'   // text, numit string, se scrie între ghilimele
const zile = 4            // număr
const vizitat = false     // valoare logică: true sau false

let contor = 0
contor = contor + 1       // permis, pentru că a fost declarat cu let

console.log(oras, zile)   // afișează în consolă: Lisabona 4
```

`console.log(...)` afișează în consolă orice i se dă, inclusiv mai multe valori odată. Este principalul instrument de verificare în această lucrare.

## 3.2 Liste și obiecte

O **listă**, numită array, ține mai multe valori în ordine, între paranteze drepte. Un **obiect** ține valori cu nume, între acolade. Cele două se combină: o listă de obiecte este forma în care se păstrează, de obicei, datele unei aplicații.

```
const orase = ['Lisabona', 'Porto', 'Kyoto']

console.log(orase.length) // 3, câte elemente are lista
console.log(orase[0])     // 'Lisabona', numărarea începe de la 0

const destinatie = { oras: 'Lisabona', tara: 'Portugalia', zile: 4 }

console.log(destinatie.oras)  // 'Lisabona', accesul la o proprietate
console.log(destinatie.zile)  // 4

const destinatii = [
  { oras: 'Lisabona', zile: 4 },
  { oras: 'Kyoto', zile: 6 },
]

console.log(destinatii[1].oras) // 'Kyoto'
```

### 3.3 Funcții

O funcție este o bucată de cod cu nume, care primește valori, numite parametri, și întoarce un rezultat. Forma modernă, folosită în toată lucrarea, se numește **arrow function**:

```
// forma cu acolade are nevoie de return
const dublu = (n) => {
  return n * 2
}

// fără acolade, rezultatul se întoarce automat
const triplu = (n) => n * 3

console.log(dublu(5))  // 10
console.log(triplu(5)) // 15
```

**Atenție:** Pentru a întoarce direct un obiect, acesta se pune în paranteze rotunde: `(d) => ({ ...d, vizitat: true })`. Fără ele, acoladele sunt înțelese ca fiind corpul funcției, iar rezultatul este `undefined`.

### 3.4 Condiții și comparații

Comparația se scrie cu trei semne egal, `===`, care verifică și valoarea, și tipul. Negarea se scrie cu semnul exclamării. Pentru o alegere scurtă între două valori se folosește operatorul ternar, `condiție ? valoare1 : valoare2`.

```
const zile = 4

console.log(zile === 4)   // true
console.log(zile !== 4)   // false
console.log(zile > 2)     // true

const vizitat = false
console.log(!vizitat)     // true, negarea

const stare = vizitat ? 'vizitat' : 'de vizitat'
console.log(stare)        // 'de vizitat'
```

### 3.5 Scurtături folosite des

| Construcție | Exemplu | Ce face |
|---|---|---|
| template literal | `` Am ${zile} zile `` | Compune un text din bucăți; valorile se pun în `${...}`, iar textul se scrie între accente grave. |
| destructuring | `const { oras, zile } = d` | Scoate două proprietăți dintr-un obiect în două variabile, dintr-o singură linie. |
| spread, pe obiect | `{ ...d, vizitat: true }` | Construiește un obiect nou, cu toate proprietățile lui `d`, dar cu `vizitat` schimbat. |
| spread, pe listă | `[...orase, 'Sevilla']` | Construiește o listă nouă, cu elementele existente plus unul în plus. |
| `??` | `x ?? 0` | Întoarce `x`, iar dacă `x` nu există, întoarce `0`. Util la contoare. |

Ultimele două merită reținute în forma lor de principiu: nu se modifică valoarea existentă, ci se construiește una nouă, plecând de la cea veche. Acesta este tiparul folosit în toată lucrarea și, mai târziu, în React.

### 3.6 Texte și numere

Textele și numerele au metode proprii, adică funcții care se apelează direct pe valoare, cu punct. Ele nu modifică valoarea inițială, ci întorc una nouă.

```
const oras = '  Lisabona  '

oras.length              // 12, câte caractere are, spațiile incluse
oras.trim()              // 'Lisabona', fără spațiile de la capete
oras.trim().toUpperCase()  // 'LISABONA'
oras.includes('Lis')     // true, conține textul dat
'Lisabona, Portugalia'.split(', ') // ['Lisabona', 'Portugalia']

const zileText = '4'     // atenție: este text, nu număr
zileText + 1             // '41', textele se lipesc
Number(zileText) + 1     // 5, după conversie

Math.round(4.6)          // 5
Math.max(4, 6, 2)        // 6
```

**Atenție:** Valorile citite dintr-un formular sau dintr-o adresă sunt întotdeauna text. Înainte de calcule se convertesc explicit, cu `Number(...)`, altfel `+` lipește textele în loc să adune.

### 3.7 Bucle și metode pe liste

Parcurgerea unei liste se poate face cu o buclă `for ... of`, care ia elementele pe rând:

```
const orase = ['Lisabona', 'Porto', 'Kyoto']

for (const oras of orase) {
  console.log(oras)
}
```

În practică, pentru liste se folosesc metodele dedicate, care spun prin numele lor ce se întâmplă și care nu modifică lista inițială. Sunt tratate pe larg în secțiunea 5, iar cele mai des întâlnite sunt:

| Metodă | Exemplu | Ce întoarce |
|---|---|---|
| `includes` | `orase.includes('Porto')` | `true` sau `false`, dacă valoarea se află în listă. |
| `indexOf` | `orase.indexOf('Porto')` | Poziția valorii, sau `-1` dacă nu există. |
| `find` | `list.find((d) => d.id === 2)` | Primul element care respectă condiția, sau `undefined`. |
| `forEach` | `orase.forEach((o) => console.log(o))` | Nimic; execută ceva pentru fiecare element. |
| `join` | `orase.join(', ')` | Un text, din toate elementele, unite cu separatorul dat. |
| `slice` | `orase.slice(0, 2)` | O listă nouă, cu o parte din elemente. |

### 3.8 Erori și depanare

Când ceva nu merge, primul loc de căutat este consola. Mesajele de eroare spun tipul problemei, fișierul și linia. Cele mai frecvente:

| Mesaj | Ce înseamnă |
|---|---|
| `ReferenceError: x is not defined` | Se folosește un nume care nu a fost declarat sau importat. |
| `TypeError: Cannot read properties of undefined` | Se cere o proprietate de la ceva care nu există, de exemplu `destinatii[9].oras` într-o listă cu 6 elemente. |
| `x is not a function` | Numele există, dar nu este funcție; adesea o greșeală de scriere. |
| `Uncaught SyntaxError` | Lipsește o paranteză, o acoladă sau o ghilimea. |

Operațiile care pot eșua, de exemplu cererile către un server, se încadrează într-un bloc `try` / `catch`: dacă apare o eroare în `try`, execuția sare în `catch`, iar programul continuă.

```
try {
  const raspuns = poateEsua()
  console.log(raspuns)
} catch (error) {
  console.error('A apărut o problemă:', error.message)
}
```

**Observație:** Pe lângă `console.log`, sunt utile `console.table(lista)`, care afișează o listă de obiecte ca tabel, și `console.error(...)`, care scrie mesajul cu roșu. Pentru probleme mai greu de urmărit, tab-ul **Sources** din DevTools permite oprirea execuției pe o linie și inspectarea valorilor.

## 4. Codul împărțit în module

Codul nu stă într-un singur fișier, ci se împarte după responsabilități: unul ține datele, altul funcțiile de prelucrare, altul comunicarea cu internetul. Un fișier nu împarte nimic cu celelalte până când nu se cere explicit: o valoare devine vizibilă în afara lui doar dacă este marcată cu `export`, iar în fișierul care o folosește trebuie adusă cu `import` (figura 4).

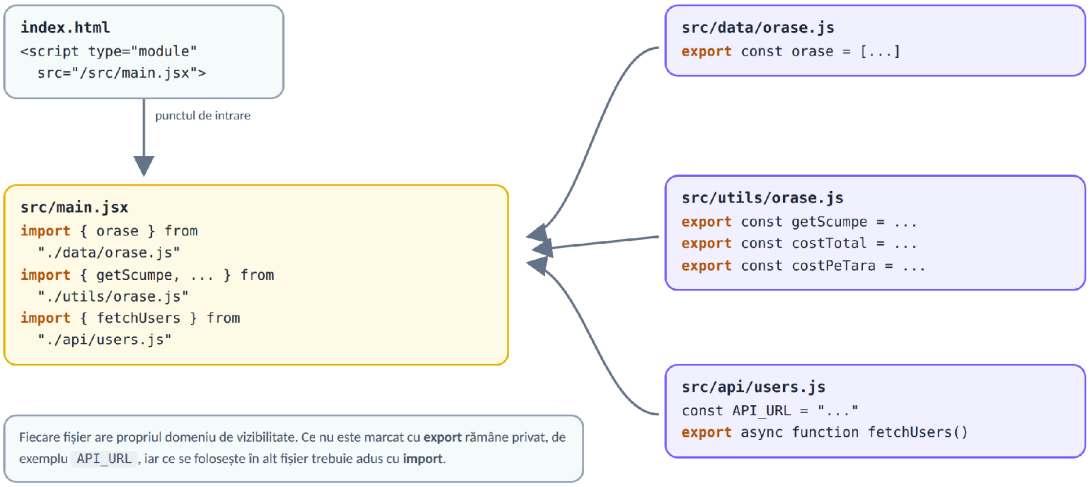



**Figura 4.** *Ce exportă fiecare fișier și ce importă main.jsx.*

```
// src/data/orase.js
export const orase = [
  { id: 1, oras: 'Madrid', tara: 'Spania', pret: 520 },
  { id: 2, oras: 'Barcelona', tara: 'Spania', pret: 480 },
  { id: 3, oras: 'Praga', tara: 'Cehia', pret: 310 },
  { id: 4, oras: 'Viena', tara: 'Austria', pret: 390 },
]

// src/main.jsx
import { orase } from './data/orase.js'

console.log(orase.length) // 4
```

Trei reguli acoperă aproape toate erorile de import: calea începe cu `./` sau `../`; extensia `.js` se scrie întotdeauna; numele dintre acolade este identic cu cel de la `export`, inclusiv literele mari și mici.

## 5. Prelucrarea unei liste

Aproape orice operație asupra unei liste intră în una dintre trei categorii: se transformă fiecare element, se păstrează doar o parte din elemente sau se obține o singură valoare din toată lista (figura 5).

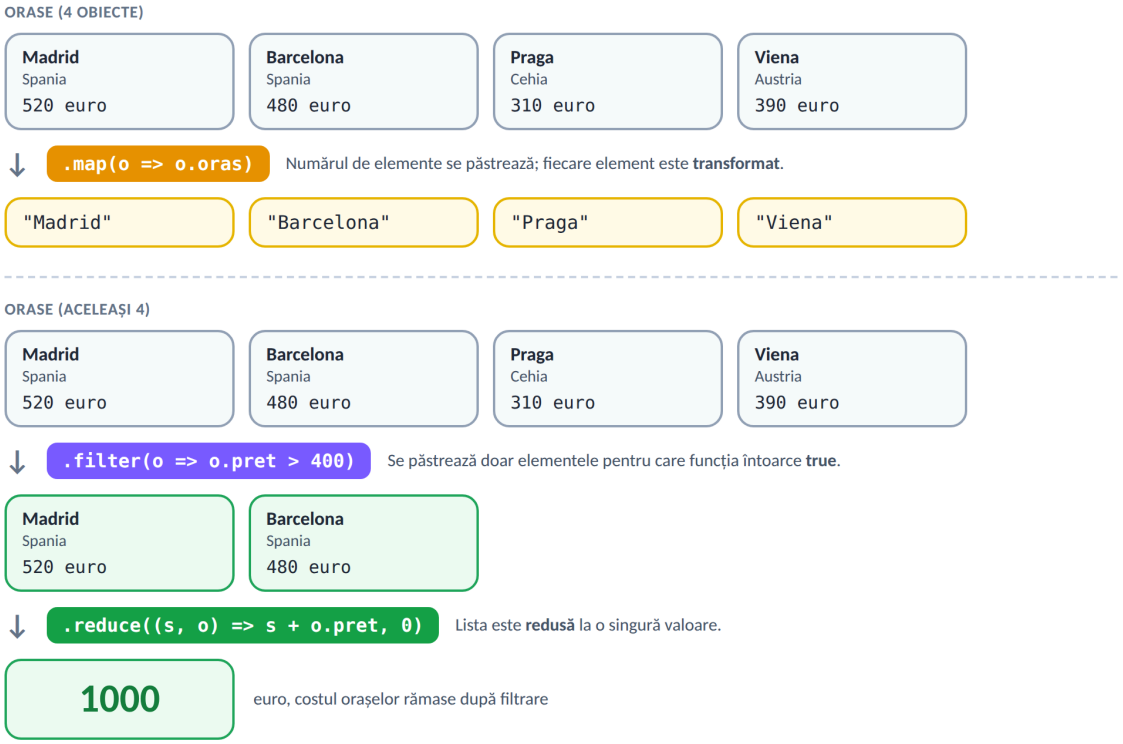



**Figura 5.** *Aceeași listă trecută prin map, filter și reduce.*

```
const numbers = [3, 8, 1]

numbers.map((n) => n * 10)              // [30, 80, 10]
numbers.filter((n) => n > 2)            // [3, 8]
numbers.reduce((sum, n) => sum + n, 0) // 12
```

| Metoda | Ce întoarce | Câte elemente are rezultatul |
|---|---|---|
| `map` | o listă nouă | exact câte are lista inițială |
| `filter` | o listă nouă | la fel sau mai puține |
| `reduce` | o singură valoare | nu este o listă |

La `reduce`, funcția primește rezultatul acumulat până în acel moment și elementul curent, iar al doilea argument este valoarea de pornire, de obicei `0` pentru sume sau `{}` pentru obiecte. Omiterea ei este cea mai frecventă greșeală: fără valoare de pornire, primul rezultat acumulat este chiar primul element al listei (figura 6).

```
orase.reduce((total, o) => total + o.pret, 0)
```

| PAS | ORAȘ | TOTAL (ÎNAINTE) | O.PRET | VALOARE ÎNTOARSĂ |
|---|---|---|---|---|
| 1 | Madrid | 0 | + 520 | **520** |
| 2 | Barcelona | 520 | + 480 | **1000** |
| 3 | Praga | 1000 | + 310 | **1310** |
| 4 | Viena | 1310 | + 390 | **1700** |

Valoarea întoarsă la un pas devine **total** la pasul următor. Valoarea de la ultimul pas (1700) este rezultatul metodei `reduce`.

**Figura 6.** *Cum acumulează reduce, pas cu pas.*

## 5.1 Exemplu rezolvat

Pe lista de orașe din secțiunea 4, funcțiile de prelucrare se grupează într-un modul propriu. Fiecare primește lista ca parametru și întoarce ceva nou, fără să modifice datele primite:

**src/utils/orase.js (fișier nou)**

```
// selectare
export const getScumpe = (list) => list.filter((o) => o.pret > 400)

// agregare
export const costTotal = (list) =>
  list.reduce((total, o) => total + o.pret, 0)

// transformare, cu destructuring în parametri
export const formatOras = ({ oras, tara, pret }) =>
  `${oras}, ${tara} - ${pret} euro (${pret > 400 ? 'scump' : 'accesibil'})`

// grupare, cu acumulator obiect și cheie calculată
export const costPeTara = (list) =>
  list.reduce((acc, { tara, pret }) =>

    ({ ...acc, [tara]: (acc[tara] ?? 0) + pret }), {})
```

**src/main.jsx**

```
import { orase } from './data/orase.js'
import { getScumpe, costTotal, formatOras, costPeTara } from './utils/orase.js'

const scumpe = getScumpe(orase)

console.log('scumpe:', scumpe.length)
console.log('cost total:', costTotal(scumpe), 'euro')
console.log(orase.map(formatOras).join('\n'))
console.log('pe țară:', costPeTara(orase))
```

**Rezultat așteptat în consolă:**

```
scumpe: 2
cost total: 1000 euro
Madrid, Spania - 520 euro (scump)
Barcelona, Spania - 480 euro (scump)
Praga, Cehia - 310 euro (accesibil)
Viena, Austria - 390 euro (accesibil)

pe țară: {Spania: 1000, Cehia: 310, Austria: 390}
```

De reținut: `formatOras` primește **un singur** oraș, nu lista; pentru un rând per element, funcția se dă lui `map`, iar rândurile se unesc cu `join`.

## 5.2 Modificarea fără stricarea listei inițiale

Metodele de până acum doar citesc datele. Urmează modificarea lor: schimbarea unei valori, adăugarea și ștergerea unui element. Regula, valabilă în toată aplicația, este că lista primită nu se modifică niciodată: se construiește una nouă, plecând de la cea veche.

**Pasul 1.** În `src/utils/orase.js` se adaugă funcția care modifică un singur element. `map` parcurge lista, iar operatorul ternar decide, pentru fiecare element: dacă este cel căutat, se construiește un obiect nou, cu spread, având `pret` schimbat; dacă nu, se întoarce elementul exact cum era.

**src/utils/orase.js (se adaugă la finalul fișierului)**

```
export const scadePret = (list, id, reducere) =>
  list.map((o) =>
    o.id === id
      ? { ...o, pret: o.pret - reducere } // element nou
      : o,                                // element refolosit
  )
```

**Pasul 2.** În `src/main.jsx` se adaugă, la final, apelul funcției pe lista deja importată, apoi se salvează:

**src/main.jsx**

```
// linia de import de la secțiunea 5.1 se completează cu scadePret
import { getScumpe, costTotal, formatOras, costPeTara,
         scadePret } from './utils/orase.js'

const reduse = scadePret(orase, 1, 40)

console.log('lista nouă:', reduse[0].pret)
console.log('lista originală:', orase[0].pret)
console.log('element refolosit:', orase[1] === reduse[1])
```

**Rezultat așteptat în consolă:**

```
lista nouă: 480
lista originală: 520
element refolosit: true
```

**Pasul 3.** Se citesc cele trei rânduri. Primul arată că modificarea a avut loc. Al doilea arată că lista inițială a rămas neatinsă; dacă ar afișa tot `480`, funcția ar fi modificat datele originale. Al treilea compară elementul neschimbat din cele două liste cu `===`, care verifică dacă este vorba de **același obiect**, nu doar de unul care arată la fel; rezultatul `true` confirmă că s-a copiat doar elementul vizat. Reprezentarea în memorie este în figura 7.

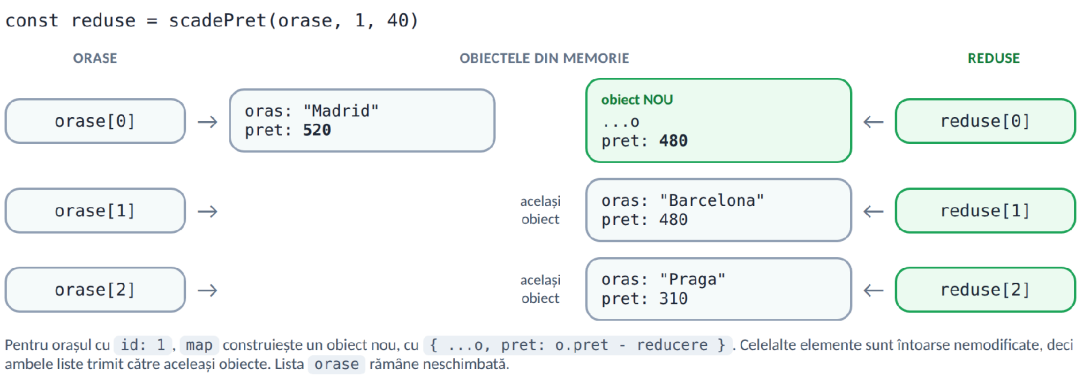



**Figura 7.** *Ce se întâmplă în memorie la o modificare de acest tip.*

Aceeași idee, pentru celelalte două operații: adăugarea înseamnă o listă nouă cu elementele vechi plus unul, iar ștergerea înseamnă o listă nouă din care lipsește un element.

```
export const adaugaOras = (list, oras) => [...list, oras]

export const stergeOras = (list, id) => list.filter((o) => o.id !== id)
```

**Atenție:** Variantele scurte, `oras.pret = 480`, `list.push(oras)` și `list.splice(...)`, modifică lista inițială. În React, interfața se actualizează doar când primește o listă nouă, deci o modificare făcută astfel nu ar fi observată: datele s-ar schimba, dar pe ecran nu s-ar întâmpla nimic. Din acest motiv tiparul se învață de acum.

## 6. Date aduse de pe internet

Datele de pe un server nu ajung instantaneu. JavaScript nu blochează pagina cât așteaptă: funcția marcată cu `async` se oprește la `await` până sosește răspunsul, iar restul paginii continuă. O cerere are doi pași, obținerea răspunsului și citirea conținutului lui (figura 8).

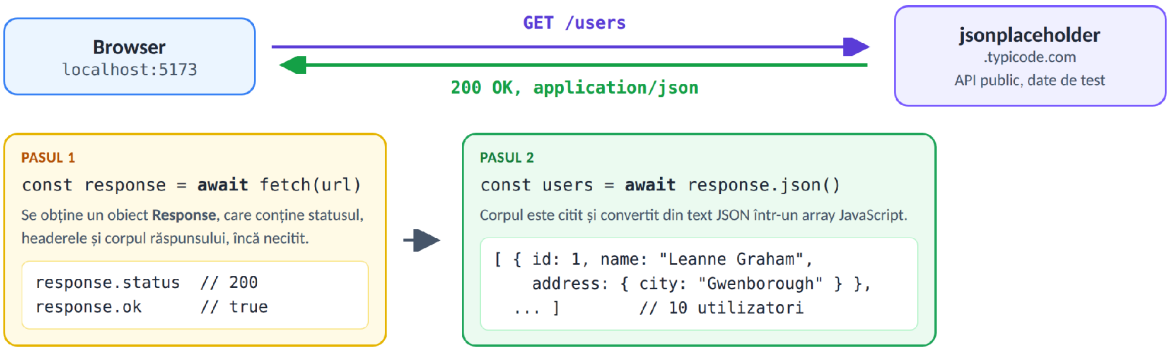



**Figura 8.** *Cei doi pași ai unei cereri fetch.*

**Atenție:** Un răspuns `404` sau `500` nu este considerat eroare de `fetch`, pentru că serverul chiar a răspuns. Se verifică `response.ok` și se aruncă eroarea manual, cu `throw`.

**Exemplu rezolvat.** Cererea se pune într-un fișier separat, iar prelucrarea rezultatului se face cu aceleași metode ca pe o listă obișnuită, pentru că exact asta este:

**src/api/users.js**

```
const API_URL = 'https://jsonplaceholder.typicode.com'

export async function fetchUsers() {
  const response = await fetch(`${API_URL}/users`)

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`)
  }

  return response.json()
}
```

**src/main.jsx**

```
import { fetchUsers } from './api/users.js'

async function loadUsers() {
  try {
    const users = await fetchUsers()
    const orase = users.map((user) => user.address.city)

    console.log('am primit', users.length, 'utilizatori')
    console.log('primele 3 orașe:', orase.slice(0, 3))
  } catch (error) {
    console.error('Nu am putut încărca datele:', error.message)
  }
}

loadUsers()
```

**Rezultat așteptat în consolă:**

```
am primit 10 utilizatori
primele 3 orașe: (3) ['Gwenborough', 'Wisokyburgh', 'McKenziehaven']
```

Metoda `slice(0, 3)` taie primele trei elemente, fără să modifice lista. De reținut: funcția care conține `await` este marcată cu `async` și trebuie și apelată, altfel nu se întâmplă nimic.

# 7. Exerciții

Exercițiile continuă proiectul **City Explorer**, de data aceasta cu date în JavaScript. Se lucrează în proiectul creat la secțiunea 2. Toate rezultatele se afișează în consola browserului, deschisă cu **F12**, tab-ul **Console**.

Codul se împarte pe fișiere astfel:

| Fișier | Ce conține |
|---|---|
| `src/data/destinatii.js` | lista de destinații, exportată |
| `src/utils/destinatii.js` | funcțiile de prelucrare, fiecare exportată separat |
| `src/api/orase.js` | funcțiile care cer date de la server |
| `src/main.jsx` | importurile, apelurile și afișarea rezultatelor |

Fișierul `src/main.jsx` crește de la un laborator la altul: codul scris azi rămâne acolo și în lucrările următoare. Ca să se știe, în consolă, ce rezultat vine din ce lucrare, fiecare mesaj începe cu `L2`, iar codul lucrării se marchează cu o linie de comentariu. În laboratorul următor prefixul devine `L3`.

```
// ===== Laboratorul 2 =====

// ... codul exercițiilor de mai jos ...

// ===== Laboratorul 3 ===== (se adaugă săptămâna viitoare)
```

### Exercițiul 1. Datele și funcțiile de prelucrare

**a) Datele.** Să se creeze fișierul `src/data/destinatii.js` și să se copieze în el, integral și exact în forma de mai jos, lista de destinații. Lista este fixată în enunț și identică pentru toată lumea, astfel încât rezultatele afișate în consolă să poată fi comparate direct cu cele date la fiecare punct:

**src/data/destinatii.js (fișier nou, se copiază integral)**

```
export const destinatii = [
  { id: '1', oras: 'Lisabona', tara: 'Portugalia', zile: 4, vizitat: false },
  { id: '2', oras: 'Cluj-Napoca', tara: 'România', zile: 2, vizitat: true },
  { id: '3', oras: 'Porto', tara: 'Portugalia', zile: 3, vizitat: false },
  { id: '4', oras: 'Brașov', tara: 'România', zile: 2, vizitat: true },
  { id: '5', oras: 'Kyoto', tara: 'Japonia', zile: 6, vizitat: false },
  { id: '6', oras: 'Tokyo', tara: 'Japonia', zile: 5, vizitat: false },
]
```

**b) Funcțiile.** Să se creeze fișierul `src/utils/destinatii.js` și să se implementeze cele șase funcții din tabel. Fiecare primește lista ca parametru, nu o importă singură, și nu modifică lista primită. Exemplele din tabel arată apelul și rezultatul exact, pe lista de destinații scrisă la punctul a).

| Funcție | Ce trebuie să întoarcă | Exemplu de apel și rezultat |
|---|---|---|
| `getOrase(list)` | Listă cu numele orașelor. Se folosește `map`. | `getOrase(destinatii)` → `['Lisabona', 'Cluj-Napoca', 'Porto', 'Brașov', 'Kyoto', 'Tokyo']` |
| `getNevizitate(list)` | Destinațiile cu `vizitat: false`. Se folosește `filter`. | `getNevizitate(destinatii).length` → `4` |

| Funcție | Ce trebuie să întoarcă | Exemplu de apel și rezultat |
|---|---|---|
| `totalZile(list)` | Suma zilelor. Se folosește `reduce`, cu valoarea de pornire `0`. | `totalZile(destinatii)` → `22` |
| `formatDestinatie(d)` | Un rând de text pentru **o singură** destinație. Se folosesc destructuring în parametri, template literal și operatorul ternar. | `formatDestinatie(destinatii[1])` → `'[x] Cluj-Napoca, România (2 zile)'` |
| `numarPeTara(list)` | Obiect cu numărul de destinații din fiecare țară. Se folosește `reduce`, cu valoarea de pornire `{}` și cheie calculată. | `numarPeTara(destinatii)` → `{Portugalia: 2, România: 2, Japonia: 2}` |
| `sortDupaZile(list)` | Lista ordonată descrescător după `zile`. `sort` modifică lista pe care este aplicată, deci se ordonează o copie: `[...list]`. | `sortDupaZile(destinatii)[0].oras` → `'Kyoto'` |

**c) Afișarea.** În `src/main.jsx` se importă lista și funcțiile, apoi se afișează, în această ordine: lista orașelor; câte destinații nu au fost vizitate; totalul zilelor pentru acestea, obținut prin înlănțuirea a două funcții; lista formatată, un rând per destinație, unită cu `join('\n')`; distribuția pe țări; orașul cu cel mai mare număr de zile.

Primele două linii se scriu astfel; restul urmează același tipar:

**src/main.jsx**

```
import { destinatii } from './data/destinatii.js'
import { getOrase, getNevizitate, totalZile } from './utils/destinatii.js'

console.log('L2 Ex. 1 - orașe:', getOrase(destinatii))
console.log('L2 Ex. 1 - nevizitate:', getNevizitate(destinatii).length)
```

**Rezultat așteptat în consolă:**

```
L2 Ex. 1 - orașe: (6) ['Lisabona', 'Cluj-Napoca', 'Porto', 'Brașov', 'Kyoto',
'Tokyo']
L2 Ex. 1 - nevizitate: 4
L2 Ex. 1 - zile de vizitat: 18
[ ] Lisabona, Portugalia (4 zile)
[x] Cluj-Napoca, România (2 zile)
[ ] Porto, Portugalia (3 zile)
[x] Brașov, România (2 zile)
[ ] Kyoto, Japonia (6 zile)
[ ] Tokyo, Japonia (5 zile)
L2 Ex. 1 - pe țară: {Portugalia: 2, România: 2, Japonia: 2}
L2 Ex. 1 - cea mai lungă vizită: Kyoto
```

### Exercițiul 2. Modificarea listei fără a o strica

Să se adauge, în același fișier `src/utils/destinatii.js`, trei funcții care întorc de fiecare dată o listă nouă. Modelul este cel din secțiunea 5.2, unde aceleași operații au fost făcute pe lista de orașe.

| Funcție | Ce trebuie să întoarcă | Exemplu de apel și rezultat |
|---|---|---|
| `marcheazaVizitat(list, id)` | Lista în care destinația cu identificatorul dat are `vizitat` inversat. Se folosesc `map`, operatorul ternar și spread. | `marcheazaVizitat(destinatii, '1')[0].vizitat` → `true`, iar `destinatii[0].vizitat` rămâne `false` |
| `adaugaDestinatie(list, d)` | Lista cu o destinație nouă la final. Se folosește spread pe listă. | `adaugaDestinatie(destinatii, noua).length` → `7`, iar `destinatii.length` rămâne `6` |
| `stergeDestinatie(list, id)` | Lista fără destinația cu identificatorul dat. Se folosește `filter`. | `stergeDestinatie(destinatii, '2').length` → `5` |

**Verificarea.** În `src/main.jsx` se completează linia de import cu cele trei funcții noi, apoi se adaugă, la final, apelurile de mai jos. Ele sunt scrise astfel încât să dovedească faptul că lista inițială nu a fost modificată:

**src/main.jsx**

```
// linia de import de la exercițiul 1 se completează cu cele trei funcții noi
import {
  getOrase, getNevizitate, totalZile, formatDestinatie, numarPeTara, sortDupaZile,
  marcheazaVizitat, adaugaDestinatie, stergeDestinatie,
} from './utils/destinatii.js'

const dupaMarcare = marcheazaVizitat(destinatii, '1')
console.log('L2 Ex. 2 - original:', destinatii[0].vizitat, '| listă nouă:',
dupaMarcare[0].vizitat)
console.log('L2 Ex. 2 - element refolosit:', destinatii[1] === dupaMarcare[1])

const noua = { id: '7', oras: 'Sevilla', tara: 'Spania', zile: 3, vizitat: false }
const dupaAdaugare = adaugaDestinatie(destinatii, noua)
console.log('L2 Ex. 2 - după adăugare: original', destinatii.length, ', listă nouă',
dupaAdaugare.length)

const dupaStergere = stergeDestinatie(dupaAdaugare, '7')
console.log('L2 Ex. 2 - după ștergere:', dupaStergere.length)
```

**Rezultat așteptat în consolă:**

```
L2 Ex. 2 - original: false | listă nouă: true
L2 Ex. 2 - element refolosit: true
L2 Ex. 2 - după adăugare: original 6 , listă nouă 7
L2 Ex. 2 - după ștergere: 6
```

Dacă prima linie afișează `true | true`, funcția a modificat lista originală, deci trebuie refăcută. Dacă a treia linie arată aceeași lungime pentru ambele liste, s-a folosit `push` în loc de spread.

## Exercițiul 3. Date de la un API public

Adresa `https://jsonplaceholder.typicode.com/users` întoarce o listă de 10 utilizatori de test. Fiecare utilizator are un obiect `address`, iar în el proprietatea `city`. Structura, pentru un singur utilizator, arată astfel:

```
{
  "id": 1,
  "name": "Leanne Graham",
  "address": { "street": "Kulas Light", "city": "Gwenborough" }
}
```

**a) Funcțiile.** Să se creeze fișierul `src/api/orase.js` cu două funcții `async`, fiecare verificând `response.ok` și aruncând `new Error(\`HTTP ${response.status}\`)` atunci când cererea nu a reușit:

| Funcție | Ce cere | Ce întoarce |
|---|---|---|
| `fetchUsers()` | adresa `/users` | lista celor 10 utilizatori |
| `fetchUserById(id)` | adresa `/users/:id`, cu identificatorul pus în adresă printr-un template literal | un singur utilizator |

**b) Afișarea.** În `src/main.jsx` se scrie o funcție `async`, cu un bloc `try` / `catch`, care se și apelează la final. În interiorul ei se afișează: câți utilizatori au fost primiți; lista orașelor lor, obținută cu `map` din `user.address.city`; câte nume de oraș au mai mult de 10 caractere, obținut cu `filter` și `length`; orașul utilizatorului cu identificatorul 1, adus separat cu a doua funcție.

Începutul funcției arată astfel:

**src/main.jsx**

```
import { fetchUsers, fetchUserById } from './api/orase.js'

async function loadOrase() {
  try {
    const users = await fetchUsers()
    console.log('L2 Ex. 3 - am primit', users.length, 'utilizatori')
    // restul afișărilor
  } catch (error) {
    console.error('L2 Ex. 3 - nu am putut încărca datele:', error.message)
  }
}

loadOrase()
```

**Rezultat așteptat în consolă:**

```
L2 Ex. 3 - am primit 10 utilizatori
L2 Ex. 3 - orașe: (10) ['Gwenborough', 'Wisokyburgh', 'McKenziehaven', 'South Elvis',
'Roscoeview',
'South Christy', 'Howemouth', 'Aliyaview', 'Bartholomebury', 'Lebsackbury']
L2 Ex. 3 - nume mai lungi de 10 caractere: 7
L2 Ex. 3 - orașul utilizatorului 1: Gwenborough
```

**c) Eroarea.** Se înlocuiește temporar apelul cu `fetchUserById(999)`, identificator care nu există. Serverul răspunde cu `404`, verificarea `response.ok` oprește execuția, iar în consolă trebuie să apară mesajul scris în `catch`, fără ca restul codului să se oprească:

**Rezultat așteptat în consolă:**

```
L2 Ex. 3 - nu am putut încărca datele: HTTP 404
```

## 8. Probleme frecvente

| Simptom | Cauză și soluție |
|---|---|
| `npm` is not recognized | Node.js nu este instalat sau VS Code a fost deschis înainte de instalare. Se instalează Node LTS și se repornește VS Code. |
| PowerShell: „running scripts is disabled” | Se execută o singură dată `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` sau se folosește Command Prompt. |
| `Missing script: "dev"` | Comanda se execută în afara directorului proiectului. Se intră cu `cd app`. |
| „Failed to resolve import” | Calea din `import` este greșită. Se verifică `./`, numele fișierului, extensia `.js` și literele mari și mici. |
| `ReferenceError: x is not defined` | Funcția există în `utils/`, dar nu a fost adăugată în lista de import din `main.jsx`. Se completează acoladele de la `import` cu numele ei. Dacă în loc de acest mesaj apare „does not provide an export named”, funcția e importată, dar lipsește `export` în fața definiției. |
| „does not provide an export named …” | Numele importat nu corespunde celui exportat, sau lipsește `export`. |
| `reduce` întoarce `NaN` sau `[object Object]…` | Lipsește valoarea de pornire, sau numele proprietății este greșit. |
| `Promise {<pending>}` în consolă | Lipsește `await`, iar funcția trebuie marcată cu `async`. |
| Arrow function care întoarce `undefined` | Obiectul returnat nu este încadrat în paranteze: `() => ({ ... })`. |

## 9. Bibliografie

**[1]** Vite Team, *Getting Started*, documentație oficială Vite, 2026. [Online]. Disponibil: https://vite.dev/guide/

**[2]** Mozilla Foundation, *JavaScript modules*, MDN Web Docs, 2026. [Online]. Disponibil: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

**[3]** Mozilla Foundation, *Array*, MDN Web Docs, 2026. [Online]. Disponibil: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

**[4]** Mozilla Foundation, *Using promises*, MDN Web Docs, 2026. [Online]. Disponibil: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

**[5]** Mozilla Foundation, *Using the Fetch API*, MDN Web Docs, 2026. [Online]. Disponibil: https://developer.mozilla.org/en-US/docs/Web/API/Fetch*API/Using*Fetch

**[6]** T. Sarkis, *JSONPlaceholder: Free Fake REST API*, 2026. [Online]. Disponibil: https://jsonplaceholder.typicode.com/